Actividad 4.2

Curso: Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 570)
Profesor: Luis Ricardo Peña Llamas
Alumno: Fernando Daniel Monroy Sánchez (A01750536)
Fecha: 15 de octubre de 2026

---

1. Introducción

Una melodía se define acústicamente como una secuencia temporal de notas discretas con alturas y duraciones determinadas. Una pregunta fundamental en la teoría de la información y el análisis de sistemas es si dicha secuencia de cambios puede considerarse y modelarse formalmente como un **proceso estocástico**.

Un proceso estocástico es una colección de variables aleatorias $\{X_t : t \in T\}$ indexadas por un conjunto temporal T. En la música tonal occidental, la sucesión de eventos sonoros no constituye ruido caótico ni tampoco una trayectoria estrictamente determinista: está gobernada por principios de consonancia física, tensión armónica y tendencias de resolución melódica. Por ende, la música constituye un proceso estocástico con una estructura definida de dependencia temporal.

El objetivo principal de esta práctica consiste en:

1. Diseñar una cadena de Markov de tiempo discreto que modele las transiciones entre notas de la escala diatónica de Re Mayor (D Major) y acordes armónicos.
2. Representar la dinámica del sistema mediante una matriz de transiciones y validar computacionalmente sus propiedades estocásticas.
3. Implementar un generador pseudoaleatorio de transiciones melódicas y acústicas.
4. Sintetizar físicamente cada tono empleando frecuencia fundamental y duración variable con suavizado de envolvente ADSR para prevenir distorsión acústica.
5. Analizar la convergencia hacia la distribución estacionaria mediante las ecuaciones de Chapman Kolmogorov.
6. Elaborar una reflexión profunda sobre la incorporación de patrones repetitivos, el aprendizaje de estilos mediante estimación empírica y las limitaciones estructurales del modelo.

(Ver Apéndice A para el código de configuración y parámetros)

Resumen del Sistema y Síntesis Acústica

Parámetro	Valor
Tonalidad base	Re Mayor (D Major)
Estados melódicos	7 notas (D4 a C#5)
Estados armónicos	4 acordes (D, G, A, Bm)
Frecuencia muestreo	44,100 Hz (Calidad CD)
Afinación	12 TET (A4 = 440 Hz)
Articulación	Senoidal con ADSR
Duraciones	4 figuras (120 BPM)

2. Desarrollo

2.1. Escala diatónica y representación del espacio de estados

Conforme a los principios matemáticos expuestos en el análisis acústico de Lemnismath (*¿Por qué tenemos 12 notas musicales?*), la física del sonido se fundamenta en ondas periódicas caracterizadas por amplitud y frecuencia. Al pulsar una cuerda o columna de aire no se emite una frecuencia aislada sino una superposición de armónicos enteros $f_k = k \cdot f_1$.

La consonancia acústica entre dos tonos depende de la coincidencia de sus espectros: la octava (razón 2:1) aporta identidad tonal y la quinta justa (razón 3:2) aporta equilibrio acústico. Al encadenar 12 quintas justas sucesivas se arriba a una discrepancia matemática respecto a 7 octavas exactas, conocida como la coma pitagórica (1.01364). Para resolver dicha incompatibilidad geométrica, el sistema de **temperamento igual de 12 tonos (12 TET)** divide la octava en 12 semitonos idénticos cuya razón de frecuencia constante es:

$$r = 2^{1/12} \approx 1.059463094$$

La frecuencia f de cualquier nota se calcula a partir de su índice MIDI n tomando como referencia universal la nota La 4 ($A_4 = 440$ Hz, MIDI 69):

$$f(n) = 440 \cdot 2^{\frac{n-69}{12}} \text{ Hz}$$

(Ver Apéndice B para el código de cálculo acústico)

Grados de la Escala Diatónica de Re Mayor

Nota	Grado	MIDI	Frecuencia (Hz)	Razón a Tónica
D4	I (Tónica)	62	293.66	1.0000
E4	ii (Supertónica)	64	329.63	1.1225
F#4	iii (Mediante)	66	369.99	1.2599
G4	IV (Subdominante)	67	392.00	1.3348
A4	V (Dominante)	69	440.00	1.4983
B4	vi (Submediante)	71	493.88	1.6818
C#5	vii° (Sensible)	73	554.37	1.8877

Nota sobre temperamento acústico: En el sistema 12 TET, la razón constante $2^{1/12}$ permite transportar melodías entre tonalidades preservando intervalos idénticos, a cambio de una desviación imperceptible de las quintas puras, asegurando estabilidad armónica completa.

2.2. Matriz de transiciones y validación estocástica (20%)

Una cadena de Markov homogénea y de tiempo discreto se formaliza como una tupla $(S, P, \pi^{(0)})$:

- $S = \{s_1, s_2, \dots, s_m\}$: Espacio discreto de estados (notas o acordes).
- $P \in \mathbb{R}^{m \times m}$: Matriz de probabilidades de transición donde $P_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$.
- $\pi^{(0)} \in \mathbb{R}^m$: Vector de distribución de probabilidad inicial.

Propiedad de Markov:
El estado futuro depende únicamente del estado inmediatamente presente, siendo condicionalmente independiente de la historia pasada:

$$P(X_{t+1} = s_j \mid X_t = s_i, \dots, X_0 = s_0) = P(X_{t+1} = s_j \mid X_t = s_i) = P_{ij}$$

Como expuso el profesor Luis Ricardo Peña Llamas en la clase del 8 de octubre mediante el ejemplo de los registros de estudio de 20 días con 11 días de estudio cada uno, el conteo frecuentista simple (11/20) resulta ciego ante la estructura temporal. Mientras que un registro proviene de una regla estocástica (70% de estudiar si se estudió hoy y 40% si no), el otro proviene de un volado independiente (50/50). El análisis de pares ordenados

de tokens adyacentes (**bigramas**) es la técnica matemática que permite estimar las probabilidades de transición y revelar la verdadera dinámica subyacente.

Las probabilidades de la matriz se diseñaron siguiendo las funciones armónicas de la música tonal:

- **Tónica (D4):** Centro de gravedad tonal. Estabilidad máxima que distribuye transiciones fluidas hacia subdominante (G4) y dominante (A4).
- **Supertónica (E4):** Tensión de paso que conduce preferentemente hacia la dominante (A4, 25%) y retorno a tónica (D4, 30%).
- **Mediante (F#4):** Color del modo mayor; conecta con subdominante (G4, 30%) y tónica (D4, 20%).
- **Subdominante (G4):** Fuerza armónica intermedia con fuerte inclinación hacia la dominante (A4, 35%).
- **Dominante (A4):** Grado de máxima tensión armónica. Manifiesta atracción imperativa de resolución hacia la tónica (D4, 50%) o cadencia rota a submediante (B4, 15%).
- **Submediante (B4):** Centro tonal relativo menor; conduce a subdominante (G4, 25%) y dominante (A4, 20%).
- **Sensible (C#5):** Grado a un semitono de la tónica superior. En conducción de voces tiene resolución obligada ascendente hacia la tónica (D4/D5), modelada aquí con 70% de probabilidad.

(Ver Apéndice C para el código de matriz y verificación)

Matriz de Probabilidades de Transición P

Estado (t)	D4	E4	F#4	G4	A4	B4	C#5
D4	0.10	0.20	0.15	0.20	0.20	0.10	0.05
E4	0.30	0.05	0.20	0.10	0.25	0.05	0.05
F#4	0.20	0.20	0.05	0.30	0.15	0.05	0.05
G4	0.15	0.10	0.25	0.05	0.35	0.05	0.05
A4	0.50	0.10	0.10	0.05	0.05	0.15	0.05
B4	0.20	0.15	0.15	0.25	0.20	0.05	0.00
C#5	0.70	0.10	0.05	0.00	0.10	0.05	0.00

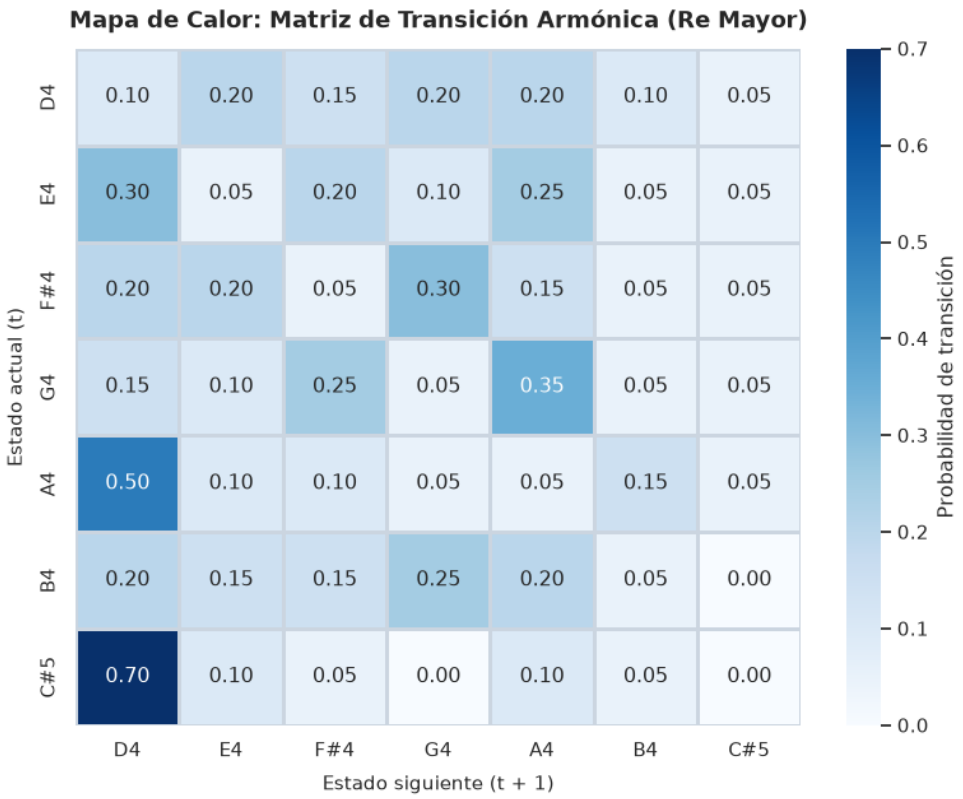
```
¿Probabilidades >= 0? : True
Suma por fila de la matriz:
[1. 1. 1. 1. 1. 1. 1.]
¿Filas suman 1.0 (allclose)? : True
```

Nota sobre la matriz estocástica: A diferencia de los modelos con distribución uniforme plana (~0.14) del prototipo web de Netlify, la matriz diseñada incorpora polaridad armónica. Cada fila representa la distribución condicional $P(X_{t+1} \mid X_t = s_i)$, sumando estrictamente 1.0.

2.3. Mapa de calor y dinámica de probabilidades

La siguiente figura ilustra visualmente la densidad de transición. Las intensidades más oscuras reflejan las resoluciones armónicas más fuertes (como la sensible C#5 hacia la tónica D4 con 0.70, y la dominante A4 hacia la tónica D4 con 0.50):

(Ver Apéndice D para el código de graficación)



Nota sobre resoluciones armónicas: El mapa de calor ilustra que la tónica actúa como sumidero probabilístico natural del sistema, atrayendo fuertemente tanto al quinto grado dominante como al séptimo grado sensible.

2.4. Evolución temporal y distribución estacionaria

Conforme a las ecuaciones de Chapman Kolmogorov vistas en clase, la distribución de probabilidad tras n transiciones se obtiene mediante potencias de la matriz estocástica:

$$\pi^{(n)} = \pi^{(0)} \cdot P^n$$

Como se demostró en el ejercicio del lenguaje de 3 palabras de Rodrigo Castellanos ("él", "gato", "duerme") presentado en clase, al simular 100 textos iniciales la distribución evoluciona en cada paso y se estabiliza tras aproximadamente 20 transiciones. Esta distribución límite es la **distribución estacionaria** π^*, que satisface el sistema invariante:

$$\pi^* \cdot P = \pi^* \quad \text{con} \quad \sum_{i=1}^{m} \pi_i = 1$$

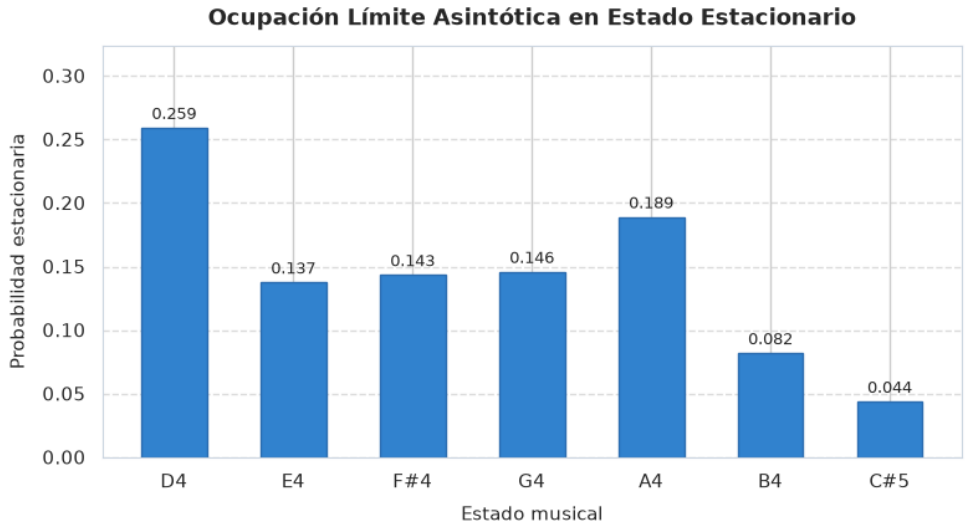
Este principio es precisamente la base matemática del algoritmo **PageRank** de Larry Page en Google, donde la importancia de una página web se determina mediante la probabilidad estacionaria de un caminante aleatorio en el grafo de hipervínculos.

Resolviendo el sistema $(P^T - I)\pi^{*T} = 0$:

(Ver Apéndice D para el cálculo de estado estacionario)

Distribución Estacionaria Asintótica

Nota	Grado	Probabilidad límite	Porcentaje
D4	I (Tónica)	0.2591	25.91%
E4	ii (Supertónica)	0.1375	13.75%
F#4	iii (Mediante)	0.1433	14.33%
G4	IV (Subdominante)	0.1457	14.57%
A4	V (Dominante)	0.1889	18.89%
B4	vi (Submediante)	0.0818	8.18%
C#5	vii° (Sensible)	0.0437	4.37%



Nota sobre convergencia asintótica: La tónica D4 (25.91%) y la dominante A4 (18.89%) concentran casi el 45% de la ocupación temporal asintótica, mientras que la sensible C#5 (4.37%) presenta la menor permanencia, validando que el modelo privilegia el reposo consonante sobre la tensión inestable.

2.5. Generador estocástico y síntesis acústica ADSR (40%)

Para determinar la transición de estados en cada instante t, se implementa el muestreo por transformada inversa de la función de distribución acumulada (CDF) condicionada a la fila del estado actual X_t.

Las funciones de generación de tonos reciben como parámetros fundamentales la **frecuencia** (f en Hz) y la **duración** (d en segundos). Para prevenir discontinuidades de fase que producen chasquidos y clics acústicos molestos, se diseñó e implementó un suavizador de amplitud **ADSR (Attack, Decay, Sustain, Release)**:

- **Attack (0.04 s):** Crecimiento lineal de amplitud de 0 a 1.
- **Decay (0.04 s):** Decaimiento suave al nivel de sostenimiento (0.85).
- **Sustain:** Mantenimiento de la energía acústica en el cuerpo de la nota.
- **Release (0.05 s):** Extinción suave a cero antes del siguiente tono.

Asimismo, atendiendo el énfasis del profesor Peña Llamas en clase de no emplear duraciones fijas sino variadas para otorgar naturalidad melódica, se muestrean estocásticamente 4 figuras rítmicas a 120 BPM:

- Negra (0.50 s, peso 50%).
- Corchea (0.25 s, peso 25%).
- Negra con puntillo (0.75 s, peso 15%).
- Blanca (1.00 s, peso 10%).

(Ver Apéndice E para el código del generador y exportación WAV)

```
Paso 0 (Inicio): Nota D4 | Frecuencia: 293.66 Hz | Duración: 1.00 s
```

```
Paso 01: D4 a F#4 | 369.99 Hz | 0.75 s
Paso 02: F#4 a G4 | 392.00 Hz | 0.50 s
Paso 03: G4 a B4 | 493.88 Hz | 0.75 s
Paso 04: B4 a F#4 | 369.99 Hz | 1.00 s
Paso 05: F#4 a E4 | 329.63 Hz | 0.75 s
Paso 06: E4 a A4 | 440.00 Hz | 0.50 s
Paso 07: A4 a D4 | 293.66 Hz | 0.75 s
Paso 08: D4 a G4 | 392.00 Hz | 0.50 s
Paso 09: G4 a F#4 | 369.99 Hz | 0.50 s
Paso 10: F#4 a E4 | 329.63 Hz | 0.50 s
Paso 11: E4 a F#4 | 369.99 Hz | 0.25 s
Paso 12: F#4 a E4 | 329.63 Hz | 0.75 s
Paso 13: E4 a A4 | 440.00 Hz | 0.50 s
Paso 14: A4 a D4 | 293.66 Hz | 1.00 s
Paso 15: D4 a A4 | 440.00 Hz | 0.25 s
Paso 16: A4 a E4 | 329.63 Hz | 1.00 s

Archivo WAV exportado exitosamente:
../export/melodia_markov_16.wav
```

A continuación se presenta el registro detallado de transiciones paso a paso:

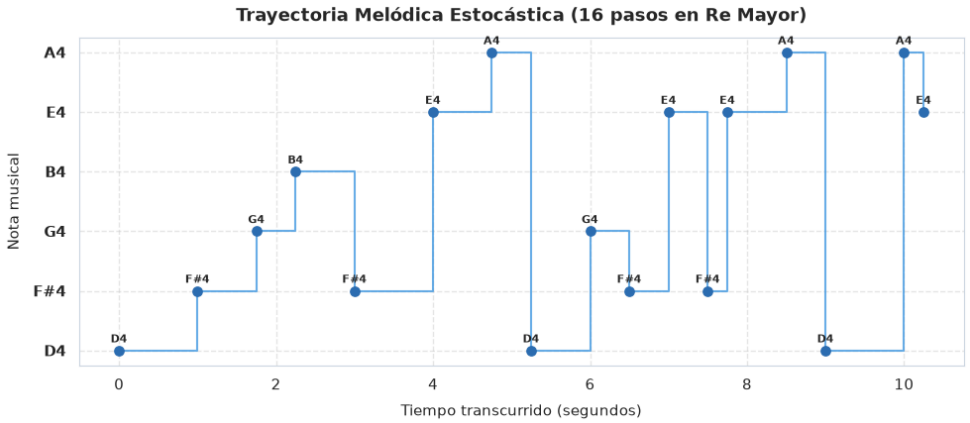
Secuencia: 16 Transiciones

Paso	Estado ant.	Nota	Frecuencia	Duración	Tiempo acum.
0	Inicio	D4	293.66	1.0	0.0
1	D4	F#4	369.99	0.75	1.0
2	F#4	G4	392.0	0.5	1.75
3	G4	B4	493.88	0.75	2.25
4	B4	F#4	369.99	1.0	3.0
5	F#4	E4	329.63	0.75	4.0
6	E4	A4	440.0	0.5	4.75
7	A4	D4	293.66	0.75	5.25
8	D4	G4	392.0	0.5	6.0
9	G4	F#4	369.99	0.5	6.5
10	F#4	E4	329.63	0.5	7.0
11	E4	F#4	369.99	0.25	7.5
12	F#4	E4	329.63	0.75	7.75
13	E4	A4	440.0	0.5	8.5
14	A4	D4	293.66	1.0	9.0
15	D4	A4	440.0	0.25	10.0
16	A4	E4	329.63	1.0	10.25

Reproducción interactiva de la melodía generada de 16 transiciones:

```
<IPython.lib.display.Audio object>
```

Trayectoria melódica en el tiempo (Piano Roll):



Nota sobre síntesis acústica: El generador produce transiciones orgánicas paso a paso. La articulación senoidal con envolvente ADSR suprime discontinuidades abruptas y genera tonos limpios con armónicos sutiles.

2.6. Composición extendida y armonización polifónica

Para apreciar la textura tonal a mayor escala, se simula una secuencia extendida de 32 pasos y una pieza polifónica a 8 compases donde una cadena markoviana gobierna la progresión armónica ($D - G - A - Bm$) mientras otra cadena genera la línea melódica solista simultáneamente:

(Ver Apéndice F para el código de armonización)

```
Melodía de 32 pasos generada con éxito:
../export/melodia_markov_32.wav
Pista polifónica generada con éxito:
../export/melodia_armonizada_8compases.wav
```

Reproductor de la pieza polifónica armonizada (8 compases completos):

```
<IPython.lib.display.Audio object>
```

Nota sobre acompañamiento armónico: El acoplamiento de cadenas concurrentes genera riqueza sonora sin elevar exponencialmente la dimensionalidad del espacio de estados individual.

3. Reflexión

Uno empieza programando estas cadenas pensando que la música va a sonar robótica o totalmente al azar, como el prototipo plano de Netlify donde todas las notas tienen casi la misma probabilidad de salir. Pero en cuanto metes pesos basados en teoría armónica, la cosa cambia por completo. Como vimos con el profe Ricardo Peña en la clase del 8 de octubre, un proceso markoviano no es lanzar una moneda al aire sin ton ni son; es capturar la estructura condicional de un sistema paso a paso. El ejemplo de los registros de estudio de 20 días que vimos en clase lo ilustra perfecto: dos alumnos pueden tener exactamente 11 días de estudio y el conteo frecuentista simple de 11 sobre 20 los ve idénticos, pero al descomponer la secuencia en bigramas descubres que uno sigue una regla clara de 70 por ciento de estudiar si estudió ayer, mientras que el otro simplemente tiraba un volado de 50 sobre 50. En la música pasa exactamente lo mismo: una escala diatónica tiene las mismas 7 notas, pero la diferencia entre una melodía coherente y puro ruido radica en la matriz de probabilidades de transición entre estados consecutivos.

Sin embargo, la música que disfrutamos culturalmente no es una caminata infinita que deambula sin rumbo; vive de la repetición deliberada de motivos, estribillos y simetrías periódicas. Una cadena de primer orden padece de amnesia temporal: solo sabe en qué nota está parada hoy, pero ignora por completo si esa nota forma parte de una frase melódica que ya se tocó dos veces o si la pieza va comenzando o terminando. Para incorporar patrones repetitivos reales tendríamos que subir a cadenas de orden superior con memoria de 3 o 4 pasos, o emplear modelos de orden variable como vimos con el modelo de lenguaje de 3 palabras de Rodrigo Castellanos (”él”, ”gato”, ”duerme”). En aquel ejercicio de clase, las probabilidades iban evolucionando en cada paso a partir del vector inicial hasta estabilizarse tras unas 20 iteraciones, exactamente igual a como el algoritmo PageRank de Larry Page calcula la relevancia de páginas en la web mediante caminantes aleatorios. No obstante, al aumentar la memoria en música el espacio de estados explota y el modelo termina repitiendo fragmentos de memoria en vez de componer material nuevo.

Por otro lado, el modelo sí tiene la capacidad de aprender la estadística local de un estilo si le suministramos un dataset de archivos MIDI de compositores barrocos o solos de jazz y calculamos las probabilidades de transición por máxima verosimilitud contando bigramas con suavizado de Laplace o Kneser Ney. Al hacer esto, la matriz absorbe los giros idiomáticos locales de inmediato, como la evitación estricta de quintas paralelas o la resolución obligada de dominante a tónica. Pero el modelo se topa con un límite estructural insalvable: carece por completo de teleología. La música tonal humana es teleológica porque plantea una tensión armónica que tarda 16 o 32 compases en resolver hacia una meta. Una cadena markoviana solo sabe qué nota conviene tocar en el siguiente medio segundo. Además, según la teoría generativa de Lerdahl y Jackendoff, el cerebro procesa la música como árboles jerárquicos de reducción similares a la sintaxis lingüística de Chomsky, con estructuras anidadas que un autómata finito regular no puede modelar, a lo que se suma que una polifonía real a 4 voces dispara el espacio conjunto a 20736 estados.

Al final, igual que ocurre con la teoría de colas M/M/1 y el dimensionamiento de cajas de cobro en Walmart que platicamos en clase, el valor de estos modelos no radica en buscar una perfección abstracta sino en construir simuladores ágiles para tomar decisiones. Al implementar duraciones rítmicas variadas y suavizado de envolvente ADSR para eliminar clics acústicos molestos en las ondas senoidales, el script genera bocetos tonales sorprendentemente musicales con un costo computacional mínimo. La industria contemporánea ya resolvió la coherencia musical global migrando hacia modelos neuronales profundos con mecanismos de atención como Transformers, pero como herramienta matemática para comprender cómo la probabilidad condicional gobierna la física del sonido, la cadena de Markov demuestra una potencia pedagógica extraordinaria.

4. Apéndices

Códigos fuente completos para garantizar reproducibilidad total.

Apéndice A: Configuración y parámetros iniciales

```
from pathlib import Path
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from great_tables import GT
from IPython.display import Audio, display

def _gt_to_latex(self):
    latex_str = self.as_latex()
    latex_str = re.sub(
        r"\\begin\{table\}(\[[^\]]*\])?",
        r"\\begin{table}[H]\n\\centering",
        latex_str
    )
    latex_str = latex_str.replace(
        r"\begin{tabular*}{\linewidth}",
        r"\begin{tabular*}{0.75\linewidth}"
    )
    latex_str = re.sub(
        r"\\fontsize\{[^}]+\}\{[^}]+\}\\selectfont",
        r"\\normalsize",
        latex_str
    )
    latex_str = re.sub(
        r"\\caption\*\{\s*\{\\large\s+([^\}]+)\}\s*\}",
        r"\\caption*{\\textbf{\\normalsize \1}}",
        latex_str
    )
    return latex_str

GT._repr_latex_ = _gt_to_latex

sns.set_theme(style="whitegrid", font_scale=0.95)
plt.rcParams["axes.edgecolor"] = "#CBD5E0"
plt.rcParams["axes.linewidth"] = 0.8

overview_data = pd.DataFrame([
    {"Parámetro": "Tonalidad base", "Valor": "Re Mayor (D Major)"},
    {"Parámetro": "Estados melódicos", "Valor": "7 notas (D4 a C#5)"},
    {"Parámetro": "Estados armónicos", "Valor": "4 acordes (D, G, A, Bm)"},
    {"Parámetro": "Frecuencia muestreo", "Valor": "44,100 Hz (Calidad CD)"},
    {"Parámetro": "Afinación", "Valor": "12 TET (A4 = 440 Hz)"},
    {"Parámetro": "Articulación", "Valor": "Senoidal con ADSR"},
    {"Parámetro": "Duraciones", "Valor": "4 figuras (120 BPM)"},
])

gt_overview = (
    GT(overview_data)
    .tab_header(title="Resumen del Sistema y Síntesis Acústica")
    .cols_align(align="left", columns=["Parámetro"])
    .cols_align(align="right", columns=["Valor"])
)
```

gt_overview

Apéndice B: Grados de la escala y frecuencias acústicas

```
from act_2.scales import D_MAJOR_NOTES_OCT4, note_to_freq, note_to_midi
import pandas as pd
from great_tables import GT

notes_data = []
for n in D_MAJOR_NOTES_OCT4:
    midi_val = note_to_midi(n)
    freq_val = note_to_freq(n)
    ratio_to_tonic = freq_val / note_to_freq("D4")
    notes_data.append({
        "Nota": n,
        "Grado": {
            "D4": "I (Tónica)",
            "E4": "ii (Supertónica)",
            "F#4": "iii (Mediante)",
            "G4": "IV (Subdominante)",
            "A4": "V (Dominante)",
            "B4": "vi (Submediante)",
            "C#5": "vii° (Sensible)"
        }[n],
        "MIDI": midi_val,
        "Frecuencia (Hz)": f"{freq_val:.2f}",
        "Razón a Tónica": f"{ratio_to_tonic:.4f}"
    })

gt_notes = (
    GT(pd.DataFrame(notes_data))
    .tab_header(title="Grados de la Escala Diatónica de Re Mayor")
    .cols_align(align="left", columns=["Nota", "Grado"])
    .cols_align(
        align="right",
        columns=["MIDI", "Frecuencia (Hz)", "Razón a Tónica"]
    )
)
gt_notes
```

Apéndice C: Matriz de transición y validación estocástica

```
import numpy as np
from great_tables import GT
from act_2.scales import (
    D_MAJOR_NOTES_OCT4,
    HARMONIC_TRANSITIONS_D_MAJOR,
)
from act_2.markov import MarkovChain

chain_notes = MarkovChain(
    D_MAJOR_NOTES_OCT4,
    HARMONIC_TRANSITIONS_D_MAJOR
)
df_matrix = chain_notes.to_dataframe()

gt_matrix = (
    GT(df_matrix.reset_index().rename(columns={"index": "Estado (t)"}))
    .tab_header(title="Matriz de Probabilidades de Transición P")
    .fmt_number(columns=D_MAJOR_NOTES_OCT4, decimals=2)
    .cols_align(align="left", columns=["Estado (t)"])
    .cols_align(align="right", columns=D_MAJOR_NOTES_OCT4)
)

row_sums = HARMONIC_TRANSITIONS_D_MAJOR.sum(axis=1)
is_non_negative = bool(np.all(HARMONIC_TRANSITIONS_D_MAJOR >= 0.0))
is_stochastic = bool(np.allclose(row_sums, 1.0, atol=1e-5))

print(f"¿Probabilidades >= 0? : {is_non_negative}")
print(f"Suma por fila de la matriz:\n{row_sums}")
print(f"¿Filas suman 1.0 (allclose)? : {is_stochastic}")
```

Apéndice D: Visualización y distribución estacionaria

```
import pandas as pd
import matplotlib.pyplot as plt
from great_tables import GT
from act_2.scales import D_MAJOR_NOTES_OCT4
from act_2.visualizer import (
    plot_transition_heatmap,
    plot_stationary_distribution,
)

fig_heatmap = plot_transition_heatmap(
    chain_notes,
    title="Mapa de Calor: Matriz de Transición Armónica (Re Mayor)"
)
plt.show()

pi_stationary = chain_notes.compute_stationary_distribution()
df_stationary = pd.DataFrame([
    {
        "Nota": note,
        "Grado": {
            "D4": "I (Tónica)",
            "E4": "ii (Supertónica)",
            "F#4": "iii (Mediante)",
            "G4": "IV (Subdominante)",
            "A4": "V (Dominante)",
            "B4": "vi (Submediante)",
            "C#5": "vii° (Sensible)"
        }[note],
        "Probabilidad límite": f"{prob:.4f}",
        "Porcentaje": f"{prob * 100:.2f}%"
    }
    for note, prob in zip(D_MAJOR_NOTES_OCT4, pi_stationary)
])

gt_stationary = (
    GT(df_stationary)
    .tab_header(title="Distribución Estacionaria Asintótica")
    .cols_align(align="left", columns=["Nota", "Grado"])
    .cols_align(
        align="right",
        columns=["Probabilidad límite", "Porcentaje"]
    )
)

fig_stat = plot_stationary_distribution(
    chain_notes,
    title="Ocupación Límite Asintótica en Estado Estacionario"
)
plt.show()
```

Apéndice E: Generador estocástico y síntesis acústica ADSR

```
from pathlib import Path
import matplotlib.pyplot as plt
from great_tables import GT
from IPython.display import Audio, display
from act_2.melody_generator import MelodyGenerator
from act_2.tones import DEFAULT_SAMPLE_RATE, export_wav
from act_2.visualizer import plot_melody_roll

export_dir = Path("export")
export_dir.mkdir(parents=True, exist_ok=True)

generator = MelodyGenerator(chain_notes)
N_TRANSICIONES = 16
audio_wave, events_list, df_melody = generator.generate_melody(
    initial_note="D4",
    n_transitions=N_TRANSICIONES,
    seed=101,
    verbose=True
)

output_wav_path = export_dir / "melodia_markov_16.wav"
export_wav(output_wav_path, audio_wave)

df_melody_display = df_melody.rename(columns={
    "Estado anterior": "Estado ant.",
    "Nota / Acorde": "Nota",
    "Frecuencia (Hz)": "Frecuencia",
    "Duración (s)": "Duración",
    "Tiempo acumulado (s)": "Tiempo acum."
})

gt_melody = (
    GT(df_melody_display)
    .tab_header(title=f"Secuencia: {N_TRANSICIONES} Transiciones")
    .cols_align(align="center", columns=["Paso"])
    .cols_align(align="left", columns=["Estado ant.", "Nota"])
    .cols_align(
        align="right",
        columns=["Frecuencia", "Duración", "Tiempo acum."]
    )
)

display(Audio(audio_wave, rate=DEFAULT_SAMPLE_RATE))

fig_roll = plot_melody_roll(
    sequence=[e.curr_state for e in events_list],
    durations=[e.duration_s for e in events_list],
    title="Trayectoria Melódica Estocástica (16 pasos en Re Mayor)"
)
plt.show()
```

Apéndice F: Composición extendida y armonización polifónica

```
from pathlib import Path
from IPython.display import Audio, display
from act_2.scales import CHORD_STATES, CHORD_TRANSITION_MATRIX
from act_2.markov import MarkovChain
from act_2.tones import DEFAULT_SAMPLE_RATE, export_wav

export_dir = Path("export")
export_dir.mkdir(parents=True, exist_ok=True)

audio_32, events_32, df_32 = generator.generate_melody(
    initial_note="D4",
    n_transitions=32,
    seed=2024,
    verbose=False
)
export_wav(export_dir / "melodia_markov_32.wav", audio_32)

chain_chords = MarkovChain(CHORD_STATES, CHORD_TRANSITION_MATRIX)
mix_audio, df_harm = generator.generate_harmonized_track(
    melody_chain=chain_notes,
    chord_chain=chain_chords,
    initial_note="D4",
    initial_chord="D",
    bars=8,
    beats_per_bar=4,
    tempo_bpm=120.0,
    seed=777
)
export_wav(export_dir / "melodia_armonizada_8compases.wav", mix_audio)
display(Audio(mix_audio, rate=DEFAULT_SAMPLE_RATE))
```